UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

FACULTAD DE INGENIERÍA

ESCUELA DE CIENCIAS Y SISTEMAS

LENGUAJES FORMALES Y DE PROGRAMACIÓN SECCIÓN: A

AUXILIAR: Carlos Javier Cox Bautista

MANUAL TÉCNICO

PROYECTO 2: TASKSCRIPT - EDITOR Y ANALIZADOR KANBAN

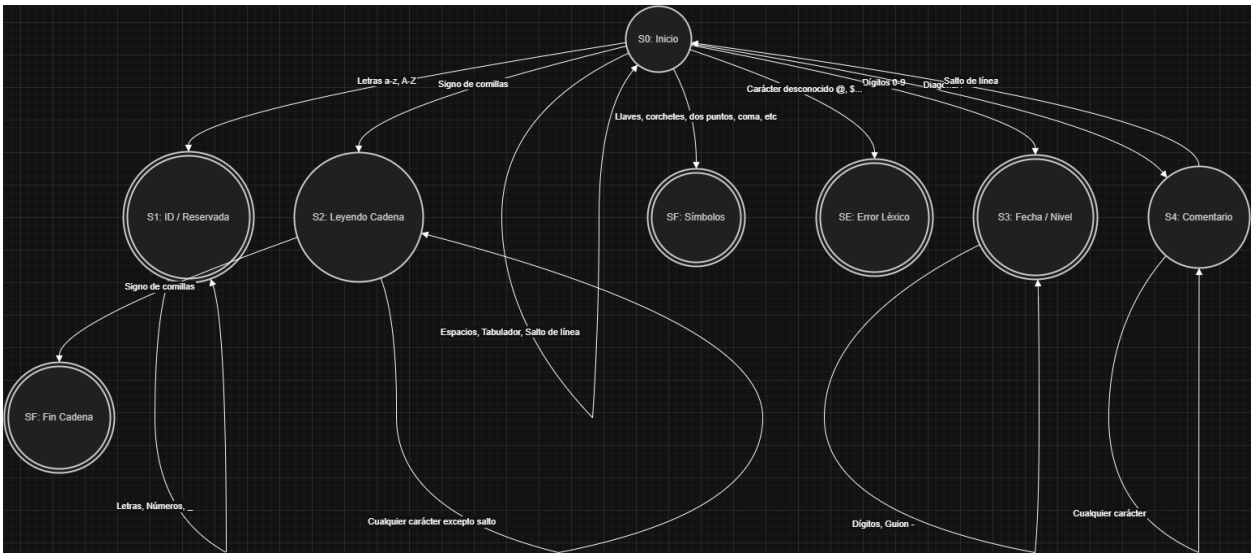
NOMBRE: Angel Sebastian Mazariegos Pérez **CARNÉ:** 202300583

Guatemala, 5 de mayo de 2026

1. Introducción y Arquitectura El compilador fue desarrollado en C++17 utilizando CMake como sistema de construcción y la biblioteca Qt para la interfaz gráfica (GUI). La arquitectura del proyecto sigue un modelo de fases estandarizado: Análisis Léxico, Análisis Sintáctico y Generación de Reportes, operando de manera secuencial y conectándose a la GUI mediante señales y *slots*.

2. Análisis Léxico El analizador léxico (lexical_analyzer) lee el código fuente carácter por carácter simulando un Autómata Finito Determinista (AFD).

- **Comportamiento:** Agrupa los caracteres en lexemas válidos y les asigna un tipo de token (TABLERO, COLUMNA, TAREA, CADENA, SIGNOS, etc.).
- **Manejo de Cadenas:** Utiliza un comportamiento *greedy*, consumiendo todos los caracteres entre comillas dobles (").
- **Errores Léxicos:** Si encuentra un carácter que no pertenece al alfabeto del lenguaje y está fuera de una cadena (ej. @ o $), genera un token de tipo ERROR, el cual es posteriormente procesado y reportado.



3. Análisis Sintáctico y Gramática Libre de Contexto (GLC) El analizador sintáctico (syntax_analyzer) implementa un modelo de Descenso Recursivo (*Recursive Descent Parser*). Valida que la secuencia de tokens proveniente del lexer cumpla con las siguientes reglas gramaticales (GLC):

<programa> ::= TABLERO CADENA { <columnas> } ;

<columnas> ::= <columna> <columnas_prima>

<columnas_prima> ::= <columna> <columnas_prima> | epsilon

<columna> ::= COLUMNA CADENA { <tareas> } ;

<tareas> ::= <tarea> <tareas_prima> | epsilon

<tareas_prima> ::= , <tarea> <tareas_prima> | epsilon

<tarea> ::= TAREA : CADENA [<atributos>]

<atributos> ::= <atributo> <atributos_prima>

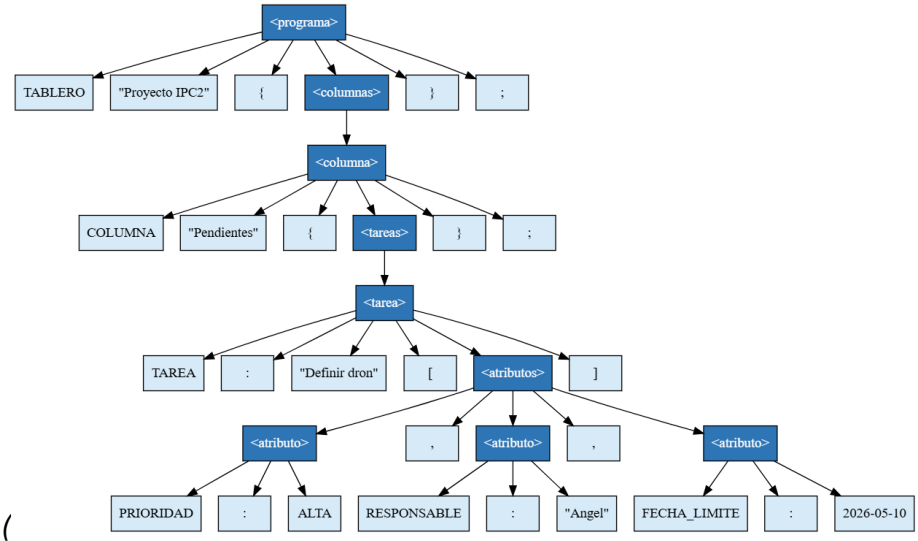
<atributos_prima> ::= , <atributo> <atributos_prima> | epsilon

<atributo> ::= PRIORIDAD : <nivel> | RESPONSABLE : CADENA | FECHA_LIMITE : FECHA

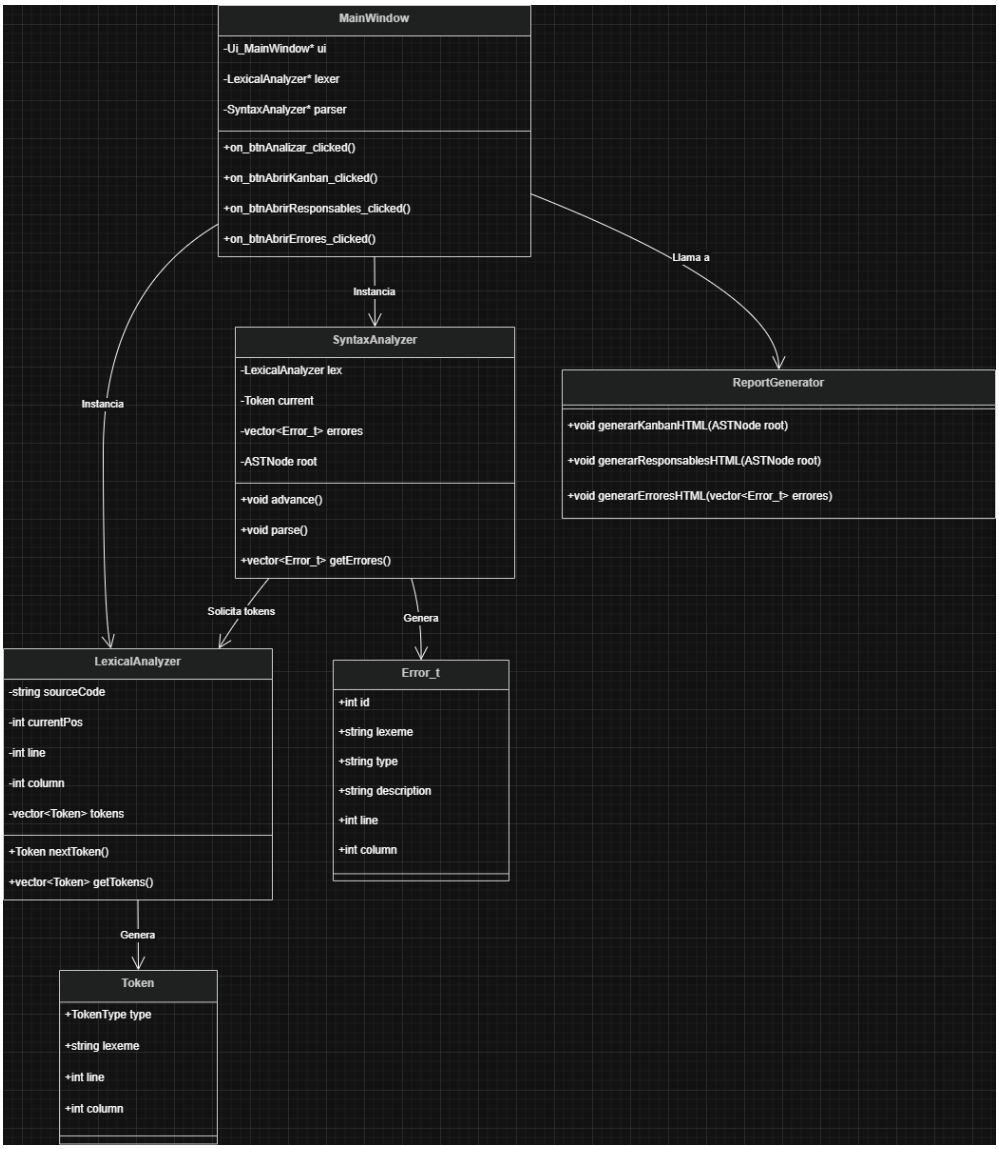
<nivel> ::= ALTA | MEDIA | BAJA

4. Recuperación de Errores (Panic Mode) El analizador sintáctico cuenta con un mecanismo robusto de recuperación de errores conocido como *Panic Mode*.

- **Filtro Léxico:** En la función advance(), si se recibe un token de tipo ERROR (carácter no reconocido), el parser lo intercepta, lo registra en la tabla como "Error Léxico", y solicita el siguiente token inmediatamente, evitando fallos sintácticos innecesarios.
- **Recuperación Sintáctica:** Al detectar un token inesperado que rompe la gramática (por ejemplo, omitir una coma o llave de cierre), el parser reporta el error y consume tokens iterativamente hasta encontrar un token de sincronización (como }, ; o la palabra reservada COLUMNA). Esto permite descartar únicamente la estructura malformada y continuar la construcción del Árbol de Derivación con el resto del código válido.

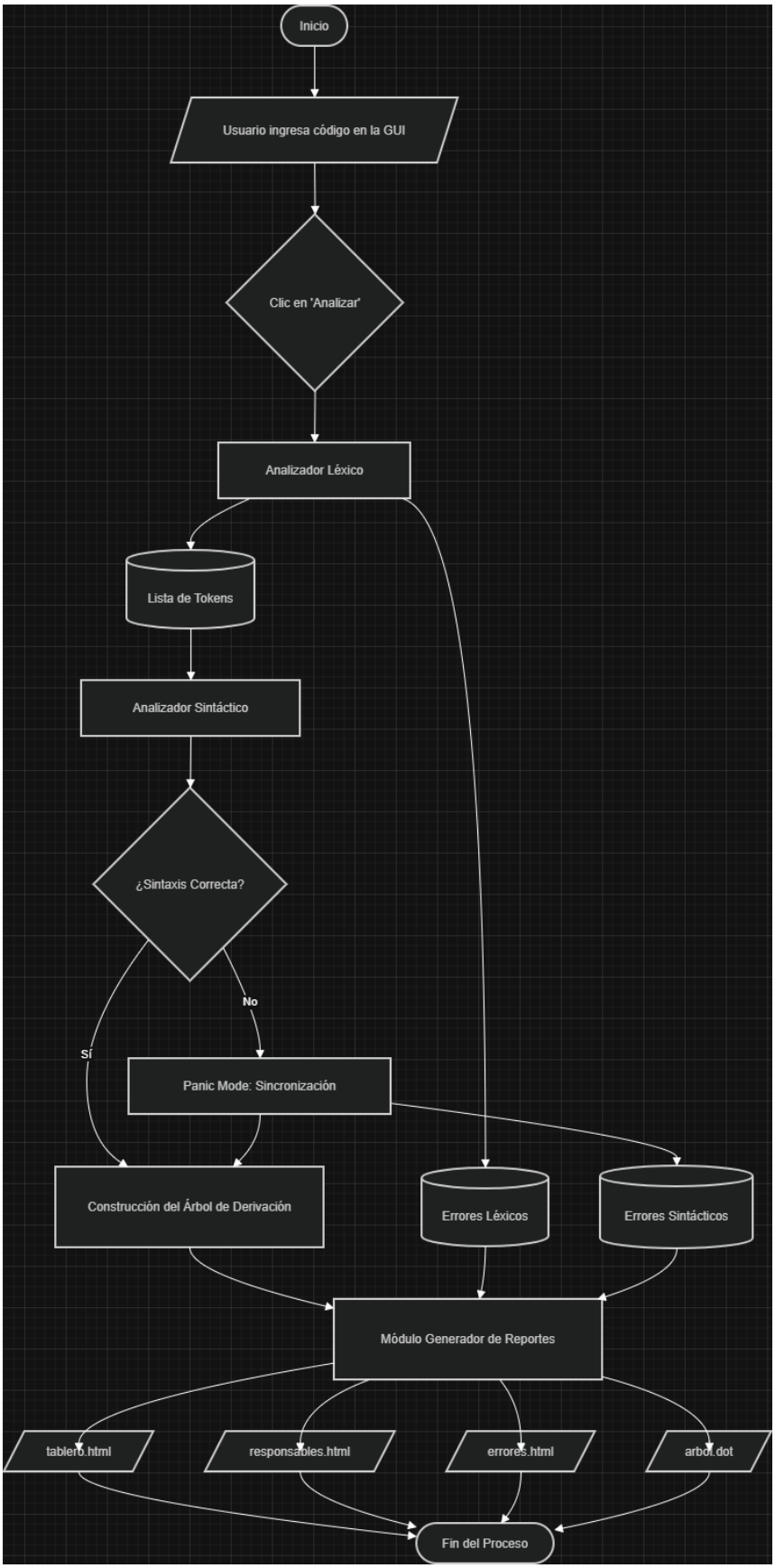

<programa>
TABLERO
"Proyecto IPC2"
{
<columnas>
}
;
<columna>
COLUMNA
"Pendientes"
{
<tareas>
}
;
<tarea>
TAREA
:
"Definir dron"
[
<atributos>
]
<atributo>
,
<atributo>
,
<atributo>
PRIORIDAD
:
ALTA
RESPONSABLE
:
"Angel"
FECHA_LIMITE
:
2026-05-10

(


MainWindow
-Ui_MainWindow* ui
-LexicalAnalyzer* lexer
-SyntaxAnalyzer* parser
+on_btnAnalizar_clicked()
+on_btnAbrirKanban_clicked()
+on_btnAbrirResponsables_clicked()
+on_btnAbrirErrores_clicked()
Instancia
Llama a
Instancia
SyntaxAnalyzer
-LexicalAnalyzer lex
-Token current
-vector<Error_t> errores
-ASTNode root
+void advance()
+void parse()
+vector<Error_t> getErrores()
ReportGenerator
+void generarKanbanHTML(ASTNode root)
+void generarResponsablesHTML(ASTNode root)
+void generarErroresHTML(vector<Error_t> errores)
Solicita tokens
Genera
LexicalAnalyzer
-string sourceCode
-int currentPos
-int line
-int column
-vector<Token> tokens
+Token nextToken()
+vector<Token> getTokens()
Error_t
+int id
+string lexeme
+string type
+string description
+int line
+int column
Genera
Token
+TokenType type
+string lexeme
+int line
+int column

5. Generación de Reportes El módulo ReportGenerator atraviesa el Árbol de Derivación (AST) generado por el parser. Utiliza manipulación de strings en C++ para estructurar archivos HTML inyectando CSS embebido para los estilos visuales de los tableros Kanban y las tablas de resumen de carga de trabajo, además de utilizar comandos de librería estándar para abrir dichos archivos mediante QDesktopServices.

Diagrama de flujo del programa:


Inicio
Usuario ingresa código en la GUI
Clic en 'Analizar'
Analizador Léxico
Lista de Tokens
Analizador Sintáctico
¿Sintaxis Correcta?
No
Sí
Panic Mode: Sincronización
Construcción del Árbol de Derivación
Errores Léxicos
Errores Sintácticos
Módulo Generador de Reportes
tablero.html
responsables.html
errores.html
arbol.dot
Fin del Proceso